

CHAPTER 07

상속과 다형성

01 상속과 다형성 소개

02 상속

03 다형성

04 is 키워드

05 클래스 자료형 변환

06 상속의 생성자

07 새도잉과 하이딩

08 하이딩과 오버라이딩

09 상속과 오버라이딩 제한

10 원도 폼: 원도 폼에서

상속과 다형성 활용하기

01 상속과 다형성 소개

- 클래스 기반의 클래스 지향 언어에서 가장 중요한 특징은 바로 상속(Inheritance)과 다형성(Polymorphism)
- 강아지와 고양이를 나타내는 클래스를 만들어보자

코드 7-1

Dog 클래스

/7장/Inheritance

```
01 class Dog
02 {
03     public int Age { get; set; }
04     public string Color { get; set; }
05
06     public Dog() { this.Age = 0; }
07
08     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
09     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
10     public void Bark() { Console.WriteLine("왈왈 짖습니다."); }
11 }
```

01 상속과 다형성 소개

- 고양이도 비슷한 방법으로 추상화

코드 7-2

Cat 클래스

/7장/Inheritance

```
01 class Cat
02 {
03     public int Age { get; set; }
04
05     public Cat() { this.Age = 0; }
06
07     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
08     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
09     public void Meow() { Console.WriteLine("냥냥 읊니다."); }
10 }
```

01 상속과 다형성 소개

- 여러 마리의 강아지와 고양이를 다루려면 배열 또는 리스트를 사용하는 것이 편리함
- [코드 7-3]은 Dog 클래스와 Cat 클래스를 생성하고 리스트에 넣은 뒤 각각의 메서드를 호출하는 예제

코드 7-3

Dog와 Cat 클래스의 인스턴스를 만들고 메서드 실행

/7장/Inheritance

```
01 static void Main(string[] args)
02 {
03     List<Dog> Dogs = new List<Dog>() { new Dog(), new Dog(), new Dog() };
04     List<Cat> Cats = new List<Cat>() { new Cat(), new Cat(), new Cat() };
05
06     foreach (var item in Dogs)
07     {
08         item.Eat();
09         item.Sleep();
10         item.Bark();
11     }
12
13     foreach (var item in Cats)
```

01 상속과 다형성 소개

```
14     {  
15         item.Eat();  
16         item.Sleep();  
17         item.Meow();  
18     }  
19 }
```

실행 결과

냥냥 먹습니다.
쿨쿨 잠을 잡니다.
왈왈 짹습니다.
냥냥 먹습니다.
쿨쿨 잠을 잡니다.
왈왈 짹습니다.
냥냥 먹습니다.
쿨쿨 잠을 잡니다.
왈왈 짹습니다.
냥냥 먹습니다.
쿨쿨 잠을 잡니다.
냥냥 읊니다.
냥냥 먹습니다.
쿨쿨 잠을 잡니다.
냥냥 읊니다.
냥냥 먹습니다.
쿨쿨 잠을 잡니다.
냥냥 읊니다.

01 상속과 다형성 소개

- [코드 7-1]부터 [코드 7-3]은 의미 없이 반복되는 부분이 굉장히 많음
- 코드의 반복을 줄인다는 것은 코드를 깔끔하게 만드는 것은 물론 생산성과 유지 보수의 측면에서 굉장히 중요한 작업
- C#에서 반복을 줄이기 위해 사용하는 방법이 바로 이번 7장의 주제인
상속과 다형성

02 상속

- 상속: 클래스 사이에 부모 자식 관계를 정의하는 작업
- 부모 클래스 하나는 자식을 여러 개 가질 수 있고, 부모 자신이 가진 것을 자식에게 물려주게 됨
- 현재 Dog 클래스와 Cat 클래스를 보면 다음과 같이 구성되어 있음

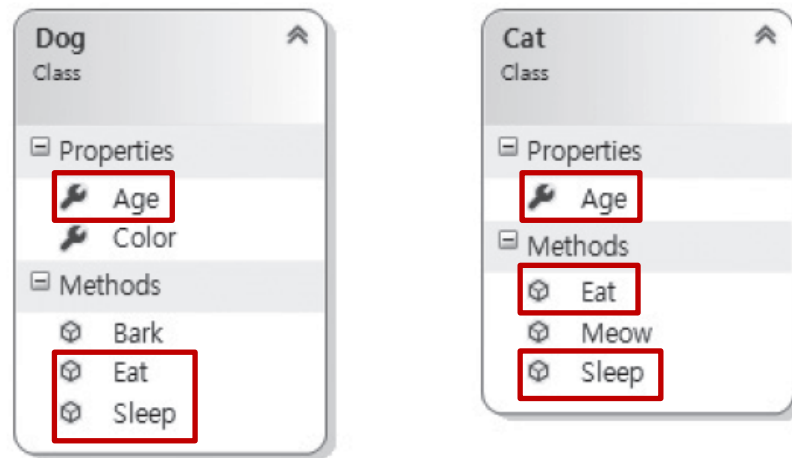


그림 7-1 분할되어 있는 클래스

02 상속

- 이 상태에서 Animal이라는 부모 클래스를 만든다면 코드를 다음과 같이 구성할 수 있게 됨

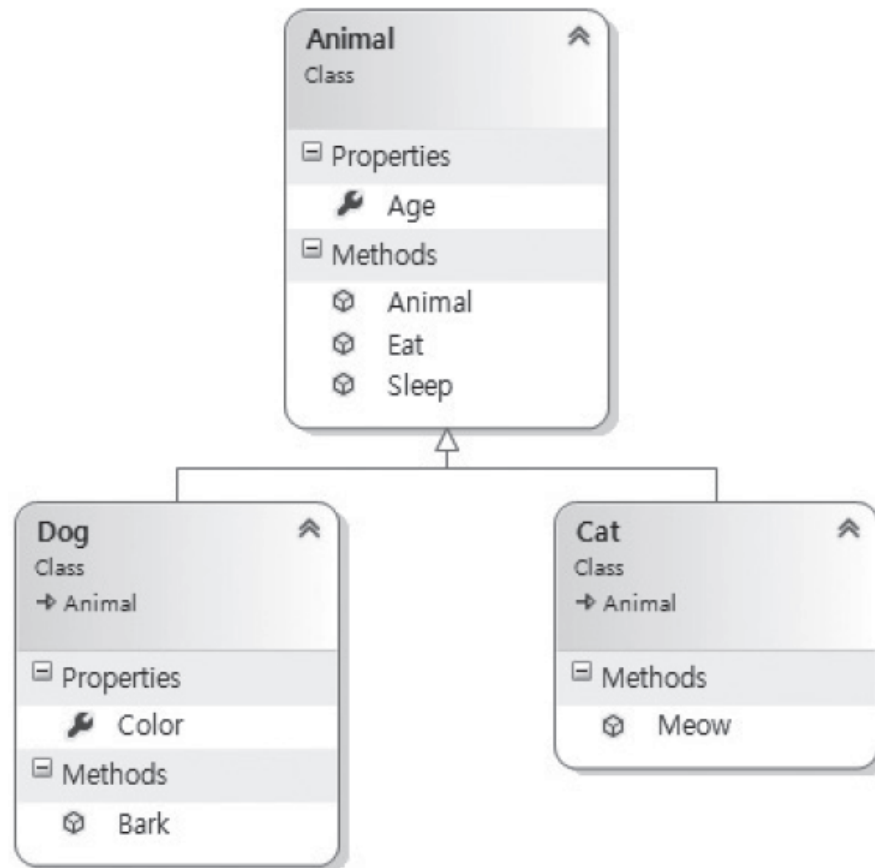
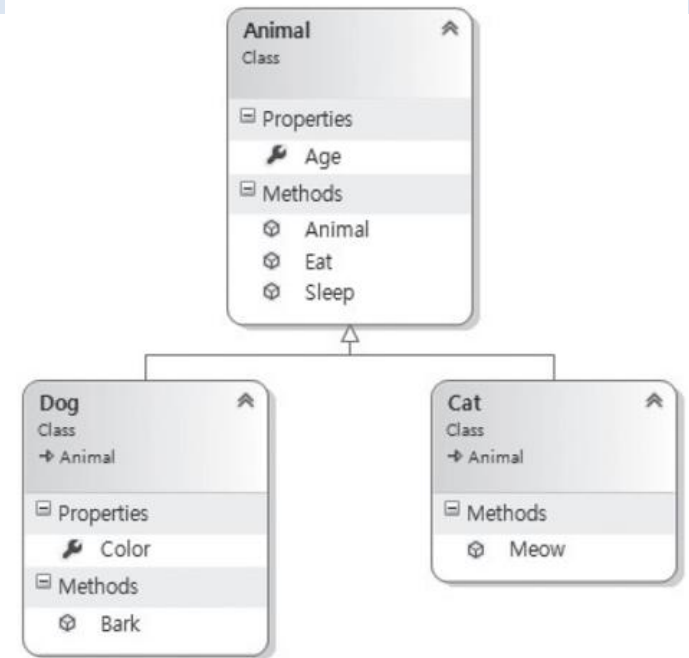


그림 7-2 상속을 사용한 구성

02 상속

- 중복되는 코드의 반복을 줄일 수 있음
- 양쪽 모두에 추가하고 싶은 멤버가 있다면 Animal 클래스만 수정하면 됨
- 이를 코드로 나타낸다면 일단 부모 클래스는 일반 클래스처럼 정의



코드 7-4

Animal 클래스

/7장/BIinheritance

```
01 class Animal
02 {
03     public int Age { get; set; }
04
05     public Animal() { this.Age = 0; }
06
07     public void Eat() { Console.WriteLine("냠냠 먹습니다."); }
08     public void Sleep() { Console.WriteLine("쿨쿨 잠을 잡니다."); }
09 }
```

02 상속

- 부모 클래스를 정의하면 자식 클래스에서 “나의 부모님은 누구입니다”라고 이야기하면 됨
- 이때는 [코드 7-5]처럼 : 기호를 사용

코드 7-5

Animal 클래스의 상속을 받는 Dog와 Cat 클래스

/7장/Inheritance

```
01 class Dog : Animal—— Animal 클래스의 상속을 받습니다.
02 {
03     public string Color { get; set; }
04
05     public void Bark() { Console.WriteLine("왈왈 짖습니다."); }
06 }
07
08 class Cat : Animal—— Animal 클래스의 상속을 받습니다.
09 {
10     public void Meow() { Console.WriteLine("냥냥 읊니다."); }
11 }
```

02 상속

- 클래스의 부모 자식 관계가 형성되면 자식 클래스는 부모 클래스의 public 또는 protected 멤버에 접근할 수 있음
- 현재 부모의 모든 멤버가 public이므로 Dog 클래스의 인스턴스를 만들면 해당 인스턴스에서 자신의 멤버는 물론 부모의 멤버에 모두 접근할 수 있음

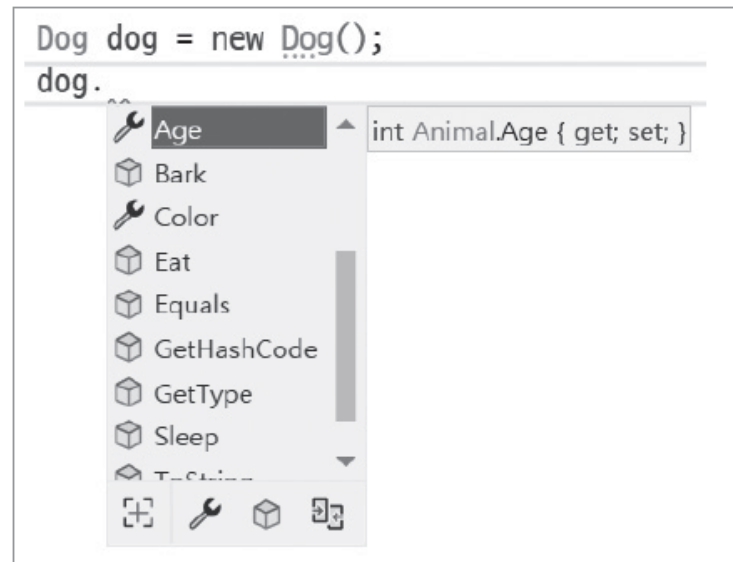


그림 7-3 Dog 클래스에서 접근할 수 있는 멤버 목록

03 다형성

- **다형성: 하나의 클래스가 여러 형태로 변환될 수 있는 성질**

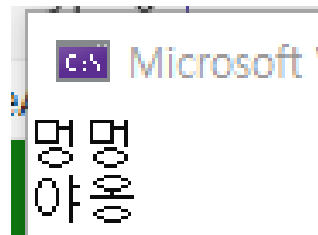
다형성 없는 경우

```
internal class Program
{
    참조 2개
    class Dog
    {
        참조 1개
        public void Sound()
        {
            Console.WriteLine("멍멍");
        }
    }

    참조 2개
    class Cat
    {
        참조 1개
        public void Sound()
        {
            Console.WriteLine("야옹");
        }
    }

    참조 0개
    static void Main(string[] args)
    {
        Dog dog = new Dog();
        Cat cat = new Cat();

        dog.Sound();
        cat.Sound();
    }
}
```



다형성 있는 경우

```
internal class Program
{
    참조 4개
    class Animal
    {
        참조 4개
        public virtual void Sound()
        {
            Console.WriteLine("동물 소리");
        }
    }

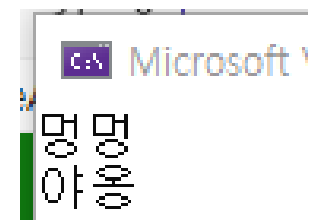
    참조 1개
    class Dog : Animal
    {
        참조 3개
        public override void Sound()
        {
            Console.WriteLine("멍멍");
        }
    }

    참조 1개
    class Cat : Animal
    {
        참조 3개
        public override void Sound()
        {
            Console.WriteLine("야옹");
        }
    }
}
```

```
static void Main(string[] args)
{
    Animal a1 = new Dog();
    Animal a2 = new Cat();

    a1.Sound();
    a2.Sound();
}
```

- Animal a1 = new Dog();
- 변수타입은 Animal인데
 - 실제객체는 Dog이라는것



03 다형성

```
Animal dog = new Dog();  
Animal cat = new Cat();
```

- 변수 dog는 외관상으로는 자료형이 Animal이지만 실제 내부에는 Dog가 들어있음
- 하나의 형태(Animal)가 여러 형태(Dog, Cat)로 동작하는 것

03 다형성

```
Dog dog = new Dog();  
Cat cat = new Cat();  
Tiger tiger = new Tiger();  
  
dog.Sound();  
cat.Sound();  
tiger.Sound();
```

```
Animal a1 = new Dog();  
Animal a2 = new Cat();  
Animal a3 = new Tiger();  
  
a1.Sound();  
a2.Sound();  
a3.Sound();
```

04 is 키워드

- 객체가 특정 타입인지 확인할때 사용
=> 이 객체가 정말 이 클래스인가요? 를 검사함

객체 is 클래스

- 결과는 true 또는 false

```
namespace ConsoleApp15
{
    참조 0개
    internal class Program
    {
        참조 3개
        class Animal { }
        참조 2개
        class Dog : Animal { }
        참조 0개
        static void Main(string[] args)
        {
            Animal animal = new Dog();

            Console.WriteLine(animal is Dog);
            Console.WriteLine(animal is Animal);
        }
    }
}
```

Microsoft Visual Studio 디버그 콘솔

True
True

04 is 키워드

```
namespace ConsoleApp15
{
    참조 0개
    internal class Program
    {
        참조 4개
        class Animal { }
        참조 1개
        class Dog : Animal { }
        참조 1개
        class Cat : Animal { }
        참조 0개
        static void Main(string[] args)
        {
            Animal animal = new Dog();

            Console.WriteLine(animal is Cat);
            Console.WriteLine(animal is Animal);
        }
    }
}
```

Microsoft Visual Studio 디버그 콘솔

False
True

05 as 키워드

객체를 특정 타입으로 **형변환(캐스팅)** 할 때 사용

=> 형변환에 성공하면 객체를 반환하고, 실패하면 예외를 발생시키지 않고 null을 반환

객체 as 타입

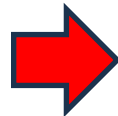
```
Dog dog = animal as Dog;
```

```
Animal animal = new Cat();  
Dog dog = (Dog)animal;
```



실제 객체가 Cat인데 Dog으로
변환하려고 해서 오류 발생

```
Animal animal = new Cat();  
Dog dog = animal as Dog;
```



dog == null
오류 발생하지 않고 null이 들어감

06 상속의 생성자

- 생성자는 인스턴스를 초기화할 때 사용
- 자식 인스턴스를 생성하면 부모가 가지고 있는 멤버 초기화를 위해 부모 생성자도 자동으로 호출됨

코드 7-18 상속했을 때 기본적인 생성자 호출 순서

/7장/ConstructorSequences

```
01 class Program
02 {
03     class Parent
04     {
05         public Parent()
06         {
07             Console.WriteLine("부모 생성자");
08         }
09     }
10
11     class Child : Parent
12     {
13         public Child()
14         {
15             Console.WriteLine("자식 생성자");
16         }
17     }
```

06 상속의 생성자

```
18
19     static void Main(string[] args)
20     {
21         Child child = new Child(); — 자식 인스턴스를 생성합니다.
22     }
23 }
```

- 코드를 실행하면 다음처럼 부모 생성자가 자동으로 먼저 호출됨
- 그리고 이어서 자식 생성자가 호출됨

실행 결과

부모 생성자

자식 생성자

base 키워드

- 만약 부모 생성자 호출을 명시적으로 지정하고 싶을 때는 [코드 7-19]처럼 base 키워드를 사용
- 생성자 메서드 뒤에 콜론을 입력하고 base()를 입력

코드 7-19 base 키워드를 사용한 생성자 지정(1)

/7장/ConstructorSequences

```
01 class Program
02 {
03     class Parent
04     {
05         public Parent() { Console.WriteLine("부모 생성자"); }
06     }
07
08     class Child : Parent
09     {
10         public Child() : base() —— base 키워드를 사용합니다.
11         {
12             Console.WriteLine("자식 생성자");
13         }
14     }
15
16     static void Main(string[] args)
17     {
18         Child child = new Child();
19     }
20 }
```

base 키워드

- "부모 클래스(기반 클래스)를 가리키는 키워드"

=> 상속 관계에서 자식 클래스가 부모 클래스의 생성자나 메서드를 사용하고 싶을 때 사용

=> 자식은 상속 받은 부모의 변수나 함수를 사용할 수 있는데 굳이 base를 이용할 필요가 있니?

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine("먹는다");
    }
}

class Dog : Animal
{
    public void Test()
    {
        Eat();           // 가능
        base.Eat();       // 이것도 가능
    }
}
```

base 키워드

```
namespace ConsoleApp15
{
    참조 0개
    internal class Program
    {
        참조 1개
        class Animal
        {
            참조 3개
            public virtual void Sound()
            {
                Console.WriteLine("동물 소리");
            }
        }

        참조 2개
        class Dog : Animal
        {
            참조 3개
            public override void Sound()
            {
                Console.WriteLine("멍멍");
            }
        }
    }
}
```

```
참조 1개
public void Test()
{
    Sound(); // 자식의 Sound()
    base.Sound(); // 부모의 Sound()
}

참조 0개
static void Main(string[] args)
{
    Dog dog = new Dog();
    dog.Test();
}
}
```

C:\ Microsoft Visu

멍멍
멍멍
동물 소리

07 새도잉과 하이딩

- 이름 재사용: 여러 변수에 같은 이름을 사용하면 이름 사이에 이름 충돌이 발생하게 되는 것
 - 오버로딩(Overloading)
 - 오버라이딩(Overriding)
 - 새도잉(Shadowing)
 - 하이딩(Hiding)

07 새도잉과 하이딩

■ 새도잉

코드 7-22 새도잉

/7장/ShadowAndHide

```
01 class Program
02 {
03     public static int number = 10;
04
05     static void Main(string[] args)
06     {
07         int number = 20;
08
09         Console.WriteLine(number);
10     }
```

static 메서드 내부에서 사용할 수 있게 static 변수로 만들었습니다.

실행 결과

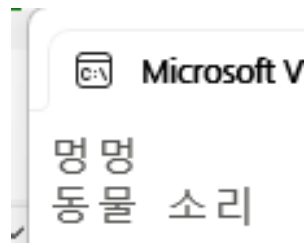
20

- 변수의 이름이 겹칠 때 자신과 가장 가까운 변수를 사용하게 되어 있음
- 즉, 메서드 내부의 변수 number를 출력하게 됨
- 새도잉(Shadowing): 특정한 영역에서 이름이 겹쳐서 다른 변수를 가림

08 하이딩과 오버라이딩

■ 하이딩

=> 부모가 가지고 있는
멤버(메서드, 변수 등)를
자식이 **가려버리는 것**
=> new 키워드 사용



```
using System;
```

```
참조 2개
```

```
class Animal
```

```
{
```

```
참조 1개
```

```
public void Sound()
```

```
{
```

```
    Console.WriteLine("동물 소리");
```

```
}
```

```
}
```

```
참조 2개
```

```
class Dog : Animal
```

```
{
```

```
참조 1개
```

```
public new void Sound() // 하이딩
```

```
{
```

```
    Console.WriteLine("멍멍");
```

```
}
```

```
}
```

```
참조 0개
```

```
class Program
```

```
{
```

```
참조 0개
```

```
static void Main()
```

```
{
```

```
    Dog dog = new Dog();
```

```
    dog.Sound();
```

```
    Animal animal = dog;
```

```
    animal.Sound();
```

```
}
```

```
}
```

08 하이딩과 오버라이딩

■ 오버라이딩

=> 부모가 가지고 있는
멤버(메서드, 변수 등)를
자식이 **덮어쓰는것**



```
using System;
```

```
참조 2개
```

```
class Animal
```

```
{
```

```
참조 3개
```

```
public virtual void Sound( )
```

```
{
```

```
    Console.WriteLine("동물 소리");
```

```
}
```

```
}
```

```
참조 2개
```

```
class Dog : Animal
```

```
{
```

```
참조 3개
```

```
public override void Sound( )
```

```
{
```

```
    Console.WriteLine("멍멍");
```

```
}
```

```
}
```

```
참조 0개
```

```
class Program
```

```
{
```

```
참조 0개
```

```
static void Main( )
```

```
{
```

```
    Dog dog = new Dog( );
```

```
    dog.Sound( );
```

```
    Animal animal = dog;
```

```
    animal.Sound( );
```

```
}
```

```
}
```

08 하이딩과 오버라이딩

구분	하이딩(new)	오버라이딩(override)
부모 메서드 존재	그대로 존재	존재하지만 자식 구현으로 대체
호출 기준	변수 타입	실제 객체 타입
<pre>Animal a = new Dog(); a.Sound()</pre>	부모 호출	자식 호출
다형성	X	O

09 상속과 오버라이딩 제한

■ sealed 키워드

- 클래스와 메서드 앞에 적용하는 키워드
- 클래스에 적용하면 절대 상속하지 말라는 의미가 되고,
- 메서드에 적용하면 더 이상 오버라이딩하지 말라는 의미

```
class Animal
{
}

sealed class Dog : Animal
{
}
```

```
class Puppy : Dog
{
}
```



```
class Animal
{
    public virtual void Sound()
    {
        Console.WriteLine("동물 소리");
    }
}
```

```
class Dog : Animal
{
    public sealed override void Sound()
    {
        Console.WriteLine("멍멍");
    }
}
```

```
class Puppy : Dog
{
    public override void Sound()
    {
        Console.WriteLine("왹왹");
    }
}
```



■ abstract 키워드

- abstract 키워드는 sealed 키워드와 반대로 무조건 상속해서 쓰라는 의미 또는 이 메서드는 반드시 오버라이딩해달라는 의미로 사용
- 상속
 - abstract 키워드를 클래스에 적용하면 반드시 상속해서 쓰라는 의미
 - 해당 클래스 자체는 인스턴스를 만들어 사용할 수 없게 됨

09 상속과 오버라이딩 제한

```
using System;
```

참조 4개

```
abstract class Animal
{
    참조 4개
    public abstract void Sound();
}
```

참조 1개

```
class Dog : Animal
{
    참조 3개
    public override void Sound()
    {
        Console.WriteLine("멍멍");
    }
}
```

참조 1개

```
class Cat : Animal
{
    참조 3개
    public override void Sound()
    {
        Console.WriteLine("야옹");
    }
}
```

참조 0개

```
class Program
{
    참조 0개
    static void Main()
    {
        Animal dog = new Dog();
        Animal cat = new Cat();

        dog.Sound();
        cat.Sound();
    }
}
```



Micro

멍멍
야옹

10 윈도 폼: 윈도 폼에서 상속과 다형성(오버로딩) 활용하기

■ 메시지 상자

- 간단한 메시지 상자 (MessageBox 클래스)를 활용해 새로운 화면을 띄우는 방법을 알아보기
- 버튼 하나만 올려두어 화면을 다음과 같이 구성

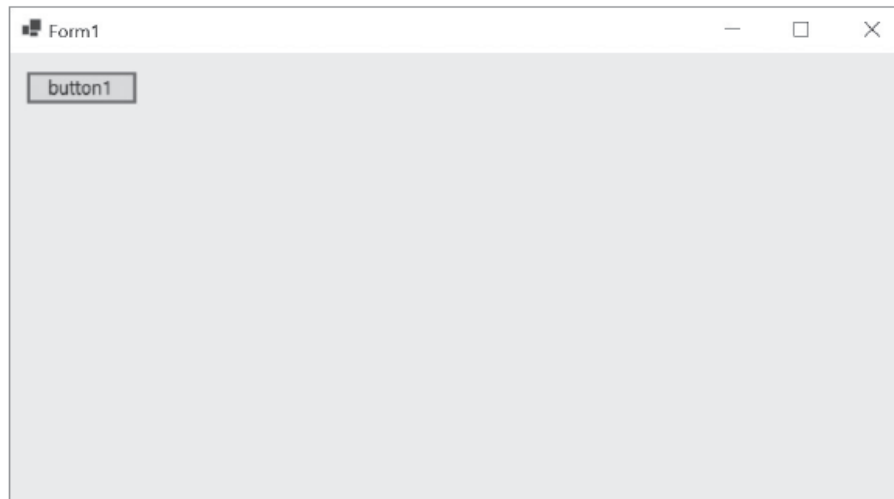


그림 7-18 화면 디자인

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

- 이어서 버튼을 더블클릭하거나 속성에서 Click 이벤트 오른쪽의 공간을 더블클릭해서 이벤트 메서드를 연결

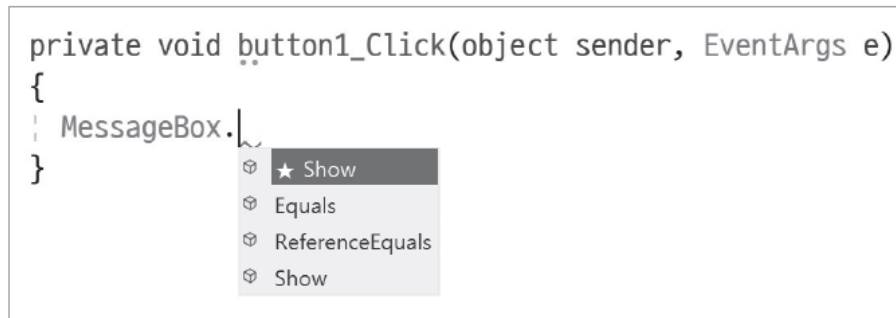


그림 7-19 MessageBox 클래스의 메서드

- static 메서드인 `Show( )`는 20가지가 넘게 오버로딩이 되어 있지만 그 중 세가지만 사용해볼 것

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

코드 7-37 MessageBox 클래스의 Show() 메서드

```
01 private void button1_Click(object sender, EventArgs e)
02 {
03     MessageBox.Show("내용");
04     MessageBox.Show("내용", "제목");
05     MessageBox.Show("내용", "제목", MessageBoxButtons.RetryCancel);
06 }
```

- 세 번째 매개변수에 넣은 MessageBoxButtons는 다음과 같은 요소들이 있고, 이 종류에 따라 다른 모양의 메시지 상자가 만들어짐

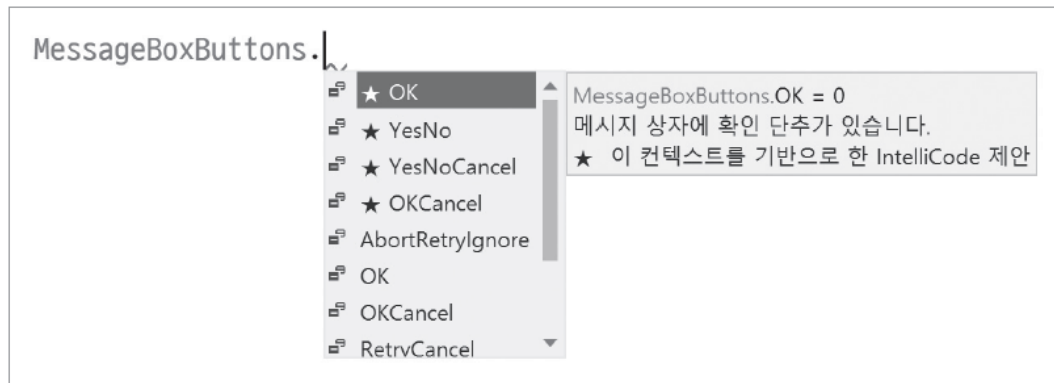


그림 7-20 MessageBoxButtons 열거자

10 윈도 폼: 윈도 폼에서 상속과 다형성 활용하기

■ 메시지 상자

- 프로젝트를 실행하고 버튼을 누르면 다음과 같은 세 종류의 메시지 상자가 차례대로 출력됨

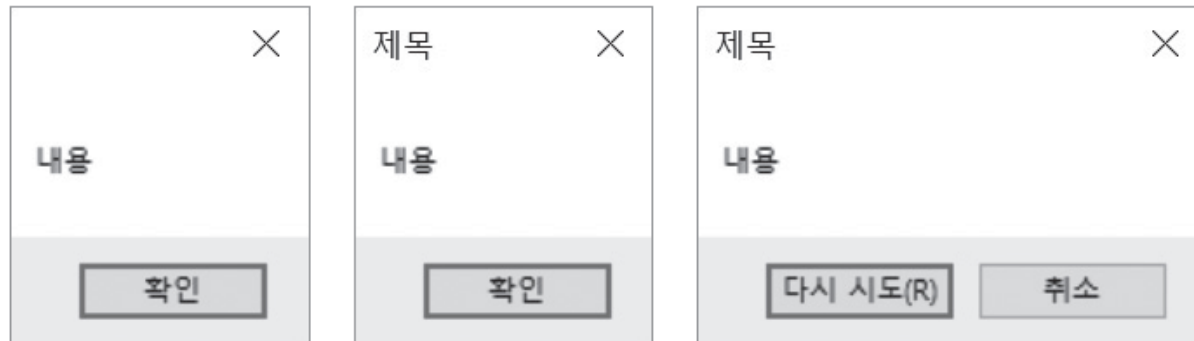


그림 7-21 메시지 상자